

FULL STACK LAB

Name:Sadhana Nautiyal

Roll Numbe:45

App setup and models

```
from django.db import models

class Fruit(models.Model):
    """
    Unordered item matrix - a simple flat collection.
    No explicit ordering field; treated as an unordered set/array.
    """
    name = models.CharField(max_length=100)
    color = models.CharField(max_length=50, blank=True)
    quantity = models.PositiveIntegerField(default=0)
    added_on = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = [] # unordered collection

    def __str__(self):
        return self.name

class Event(models.Model):
    name = models.CharField(max_length=150)
    date = models.DateField()

    def __str__(self):
        return self.name

class SelectedEventStudent(models.Model):
    """
    Ordered indices collection - students selected for an event,
    with an explicit position/index to preserve order.
    """
    event = models.ForeignKey(Event, on_delete=models.CASCADE,
related_name='selected_students')
    student_name = models.CharField(max_length=150)
    roll_number = models.CharField(max_length=50, blank=True)
    index_position = models.PositiveIntegerField(help_text="Explicit order index within
the event")

    class Meta:
        ordering = ['index_position']

    def __str__(self):
        return f"{self.student_name} ({self.index_position}) - {self.event.name}"
```

Migrations were generated and applied using:

```
python manage.py makemigrations collections_app
python manage.py migrate
```

OUTPUT

```
$ python manage.py makemigrations collections_app
Migrations for 'collections_app':
  collections_app/migrations/0001_initial.py
    - Create model Event
    - Create model Fruit
    - Create model SelectedEventStudent

$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, collections_app, contenttypes, sessions
Running migrations:
  Applying collections_app.0001_initial... OK
```

7.1 FALLBACK LOGIC FOR EMPTY ARRAYS

Views were written to pass the Fruit and SelectedEventStudent querysets to their templates. Inside the templates, {% if %} blocks check both the truthiness of the queryset and an explicit count, so an alert is shown whenever the underlying array is empty.

```
from django.shortcuts import render, get_object_or_404
from .models import Fruit, Event, SelectedEventStudent

def fruit_list_view(request):
    fruits = Fruit.objects.all()
    context = {
        'fruits': fruits,
        'fruits_count': fruits.count(),
    }
    return render(request, 'collections_app/fruit_list.html', context)

def event_student_list_view(request, event_id):
    event = get_object_or_404(Event, pk=event_id)
    students =
SelectedEventStudent.objects.filter(event=event).order_by('index_position')
    context = {
        'event': event,
        'students': students,
        'students_count': students.count(),
    }
    return render(request, 'collections_app/event_student_list.html', context)
```

Template fallback (fruit_list.html):

```
{% if fruits and fruits_count > 0 %}
<table border="1" cellpadding="6">
  <thead>
```

```

        <tr><th>Name</th><th>Color</th><th>Quantity</th></tr>
    </thead>
    <tbody>
        {% for fruit in fruits %}
        <tr>
            <td>{{ fruit.name }}</td>
            <td>{{ fruit.color|default:"N/A" }}</td>
            <td>{{ fruit.quantity }}</td>
        </tr>
        {% endfor %}
    </tbody>
</table>
{% else %}
    <div class="alert alert-warning">
        No fruits found. The array is currently empty.
        Please add items to the collection.
    </div>
{% endif %}

```

The same pattern was applied to event_student_list.html for the ordered student collection, using an alert-danger style block instead.

FRUIT LIST DATA(CURRENT)

Sample data entered via the admin panel, rendered at /fruits/:

Name	Color	Quantity
Apple	Red	40
Banana	Yellow	25
Mango	Orange	60
Grapes	Green	15

FRUIT LIST(EMPTY)

When the Fruit table has no rows, the fallback block renders the alert instead of the table:

NO fruits found. The array is currently empty. Please add items to the collection.

SELECTED STUDENTS(EMPTY)

Visiting /events/1/students/ for an event with no selections yet:

NO students have been selected for this event yet. The ordered list is empty.

8.2 DYNAMIC TABLE SORTING

Sorting- using GET parameters (sort and dir)

```

ALLOWED_FRUIT_SORT_FIELDS = {'name', 'color', 'quantity', 'added_on'}
ALLOWED_STUDENT_SORT_FIELDS = {'student_name', 'roll_number', 'index_position'}

```

```
def _resolve_sort(request, allowed_fields, default_field):
    sort_field = request.GET.get('sort', default_field)
    direction = request.GET.get('dir', 'asc')

    if sort_field not in allowed_fields:
        sort_field = default_field
    if direction not in ('asc', 'desc'):
        direction = 'asc'

    order_by = sort_field if direction == 'asc' else f'--{sort_field}'
    return order_by, sort_field, direction
```

Each view calls this helper before filtering the queryset, and the resulting sort state is passed to the template so column headers can link to the next sort direction:

```
<th>
    <a href="?sort=name&dir={% if current_sort == 'name' %}{{ next_dir }}{%
else %}asc{% endif %}">
        Name {% if current_sort == 'name' %}({{ current_dir }}){% endif %}
    </a>
</th>
```

OUTPUT-

Requesting `/fruits/?sort=quantity&dir=desc` returns the same rows ordered by quantity, descending:

Name	Color	Quantity
Mango	Orange	60
Apple	Red	40
Banana	Yellow	25
Grapes	Green	15

Requesting `/events/1/students/?sort=student_name&dir=asc` for the Selected Event Students collection returns:

#	Student Name	Roll No.
2	Aditi Verma	COER-102
4	Karan Mehta	COER-104
1	Riya Sharma	COER-101
3	Sahil Nair	COER-103

Note that the # column no longer matches the visual row order because the records are now sorted alphabetically by `student_name` rather than by `index_position` — this confirms sorting is being applied on the requested field rather than falling back to default ordering.

7.3 SEARCH FORM WITH BACKEND FILTERING

```
from django.db.models import Q

def fruit_list_view(request):
    order_by, current_sort, current_dir = _resolve_sort(
        request, ALLOWED_FRUIT_SORT_FIELDS, default_field='name'
    )

    query = request.GET.get('q', '').strip()
    fruits = Fruit.objects.all()

    if query:
        fruits = fruits.filter(
            Q(name__icontains=query) | Q(color__icontains=query)
        )

    fruits = fruits.order_by(order_by)

    context = {
        'fruits': fruits,
        'fruits_count': fruits.count(),
        'current_sort': current_sort,
        'current_dir': current_dir,
        'next_dir': 'desc' if current_dir == 'asc' else 'asc',
        'query': query,
    }
    return render(request, 'collections_app/fruit_list.html', context)
```

Search form markup:

```
<form method="get" action="{% url 'collections_app:fruit_list' %}">
    <input type="text" name="q" placeholder="Search by name or color..."
        value="{{ query|default:'' }}">
    <input type="hidden" name="sort" value="{{ current_sort }}">
    <input type="hidden" name="dir" value="{{ current_dir }}">
    <button type="submit">Search</button>
    {% if query %}
        <a href="{% url 'collections_app:fruit_list' %}">Clear</a>
    {% endif %}
</form>
```

The empty-state alert from Task 7.1 was extended to distinguish between a genuinely empty collection and a search that returned no matches:

```
{% else %}
    <div class="alert alert-warning">
        {% if query %}
            No fruits matched your search for "{{ query }}".
        {% else %}
            No fruits found. The array is currently empty.
        {% endif %}
    </div>
{% endif %}
```

The same search-and-filter logic was applied to `event_student_list_view`, filtering on `student_name` and `roll_number`.

OUTPUT

Requesting `/fruits/?q=an` (matching "Mango" and "Banana") returns a filtered subset:

Name	Color	Quantity
Banana	Yellow	25
Mango	Orange	60

Requesting a search term with no matches, `/fruits/?q=kiwi`, correctly falls through to the empty-state branch with the search-specific message:

NO fruits matched our search for 'Kiwi'